Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

In this experiment the CIFAR-10 dataset is classified using an ImageNet-pretrained VGG16 backbone, first with a frozen convolutional base and then with the last convolution block fine tuned. LeNet-5 and AlexNet are additionally trained from scratch, and InceptionV3 (GoogleNet) and ResNet50 are run through the same transfer-learning pipeline, so that the comparison table contains measured numbers rather than textbook values.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

These are the reference values for the full ImageNet models. The parameter counts measured in this experiment (Section 18.2) differ, because the classifier heads are replaced by a 10-class head and the input resolution is smaller.

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

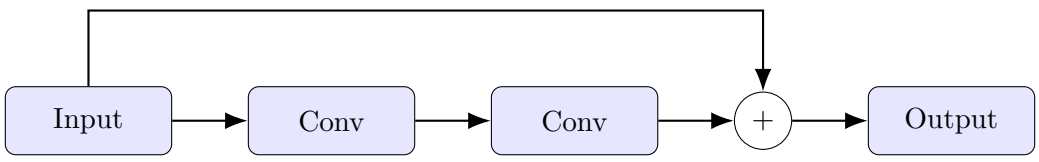
$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping



Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

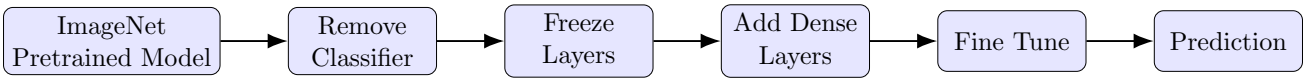
- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.



Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

The 50,000 training images are split into 45,000 images for training and 5,000 held-out images for validation, so that the validation curves are not computed on data the model has been fitted on. The 10,000 test images are used only for the final evaluation.

15. Experimental Procedure

15.0 Environment and Configuration

The experiment was implemented in TensorFlow 2.20 / Keras and executed on a Google Colab GPU runtime. Mixed precision (`mixed_float16`) was enabled so that the convolution arithmetic runs in half precision while the master weights stay in float32. A fixed random seed (42) is set for reproducibility. The setup output is shown in Figure **??**.

Parameter	Value
Backbone	VGG16 (ImageNet weights)
Input size fed to the backbone	$64 \times 64 \times 3$ (resized from 32×32)
Batch size	32
Learning rate (Task 3)	0.001
Learning rate (Task 4)	0.0001
Epochs	10 frozen + 5 fine tuning
Dense units in the head	256 (ReLU) + Dropout 0.3
Optimizer	Adam
Loss function	Categorical Cross Entropy

15.1 Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range [0,1].
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Labels are one-hot encoded so that categorical cross entropy can be used. The printed dimensions are shown in Figure **??** and the ten sample images in Figure 1.

15.2 Task 2: Transfer Learning

VGG16 was selected as the pretrained backbone. The following steps were performed.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer (`include_top=False`).

3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with 256 units and ReLU activation (with Dropout 0.3).
6. Add the output layer with 10 units and Softmax activation.

Two implementation details matter here.

- **Resizing is a layer, not a preprocessing step.** CIFAR-10 images are 32×32, which is too small for an ImageNet backbone — after five pooling stages the feature map would collapse to a single pixel. A `Resizing` layer inside the model upsamples every image to 64×64 on the GPU, instead of resizing on the CPU inside the input pipeline. This removes what would otherwise be the slowest stage of the run.
- **Model-specific preprocessing.** The Task-1 normalisation to $[0, 1]$ is undone with a `Rescaling` layer (factor 255), and VGG16's own `preprocess_input` function is then applied, because the pretrained weights expect the exact channel statistics used during ImageNet training. Feeding $[0, 1]$ inputs to an ImageNet backbone silently degrades the features.

The resulting model has 14,848,586 total parameters, of which only 133,898 (the classifier head) are trainable while the base is frozen. The model summary is shown in Figure **??**.

15.3 Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Cached feature extraction. While the convolutional base is frozen and no data augmentation is used, base(x) is a fixed function of each image: it produces the identical 512-dimensional vector in every epoch. The base is therefore run *once* over the training, validation and test sets, the pooled feature vectors are cached, and the Dense head is trained on those cached vectors for the full 10 epochs at batch size 32. The gradients, the weights and the learning curves are mathematically identical to training the whole model for 10 epochs; only the redundant recomputation of the frozen base is removed. Feature extraction took 28.3 s and head training 51.6 s, for a total of 79.8 s. The training log is shown in Figure **??** and the curves in Figure 2.

Evaluated end-to-end on the raw test images, the frozen-base model reaches a test accuracy of **0.7560**.

15.4 Task 4: Fine Tuning

1. Unfreeze the last convolution block (`block5`, i.e. layer 15 onward: 4 unfrozen layers, 15 still frozen). Trainable parameters rise from 133,898 to 7,213,322.
2. Train end-to-end for another 5 epochs.
3. Compare the classification accuracy before and after fine tuning.

The learning rate is divided by ten ($0.001 \rightarrow 0.0001$) for this phase. At the full learning rate the large gradients arriving from the freshly trained head would overwrite the pretrained ImageNet filters before they can be usefully adapted — this is the single most common mistake when fine tuning. Fine tuning took 88.9 s, giving a total training time of 168.7 s.

Stage	Test Accuracy
Before fine tuning (frozen base)	0.7560
After fine tuning (block5 unfrozen)	0.8533
Improvement	+9.73 percentage points

The fine-tuning log is shown in Figure **??**, the curves in Figure 3, and the combined 15-epoch view in Figure 4.

15.5 Task 5: Model Evaluation

The fine-tuned model was evaluated on the 10,000 test images using accuracy, precision, recall, F1-score, a confusion matrix and a per-class classification report. Precision, recall and F1 are reported as weighted averages over the ten classes. The console output is shown in Figure **??**.

Metric (weighted)	Value
Accuracy	0.8533
Precision	0.8581
Recall	0.8533
F1-score	0.8527
Misclassified images	1467 / 10000

Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.8948	0.8420	0.8676	1000
automobile	0.9648	0.8780	0.9194	1000
bird	0.8084	0.8480	0.8277	1000
cat	0.8076	0.6130	0.6970	1000
deer	0.7939	0.8780	0.8338	1000
dog	0.7103	0.8680	0.7813	1000
frog	0.9236	0.8710	0.8966	1000
horse	0.8638	0.9200	0.8910	1000
ship	0.8811	0.9340	0.9068	1000
truck	0.9323	0.8810	0.9059	1000
accuracy			**0.8533**	10000
macro avg	0.8581	0.8533	0.8527	10000
weighted avg	0.8581	0.8533	0.8527	10000

The three weakest classes are **cat** (0.613), **airplane** (0.842) and **bird** (0.848). The screenshot of the report is Figure **??**, the confusion matrix Figure 5, and a sample of the failures Figure 6.

16. Hyperparameter Study

The manual asks for a comparison over the following settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

The full cross-product is 96 runs, so a **one-factor-at-a-time** study was carried out around the baseline configuration (LR 0.001, batch 32, 10 epochs, Adam, 256 Dense units, all layers frozen): one hyperparameter is changed at a time and everything else is held fixed. Every "all frozen" variant reuses the cached features, so the whole study costs seconds rather than hours. The *Frozen = Partial* row is the Task-4 fine-tuned model, which is exactly that configuration.

Variant	LR	Batch	Epochs	Optimizer	Dense	Frozen	Val Acc (%)	Test Acc (%)	Time (s)
Baseline	0.0010	32	10	adam	256	All	76.92	75.80	49.0
LR = 0.0001	0.0001	32	10	adam	256	All	76.52	75.79	50.9
Batch = 16	0.0010	16	10	adam	256	All	76.62	75.04	89.8
Batch = 64	0.0010	64	10	adam	256	All	76.70	76.60	25.6
Epochs = 20	0.0010	32	20	adam	256	All	76.66	76.11	85.8
Optimizer = SGD	0.0010	32	10	sgd	256	All	76.24	75.50	43.8
Dense = 128	0.0010	32	10	adam	128	All	76.50	76.03	46.0
Frozen = Partial (fine-tuned)	0.0001	32	15	adam	256	Partial	**85.76**	**85.33**	168.7

The dataframe screenshot is Figure **??** and the bar chart is Figure 7.

17. Source Code

Dataset: CIFAR-10 The Source Code of this experiment will be present in the following GitHub repository: https://github.com/D-Rockie/Deep-Learning-Lab-2026.git

18. Mandatory Plots and Inferences

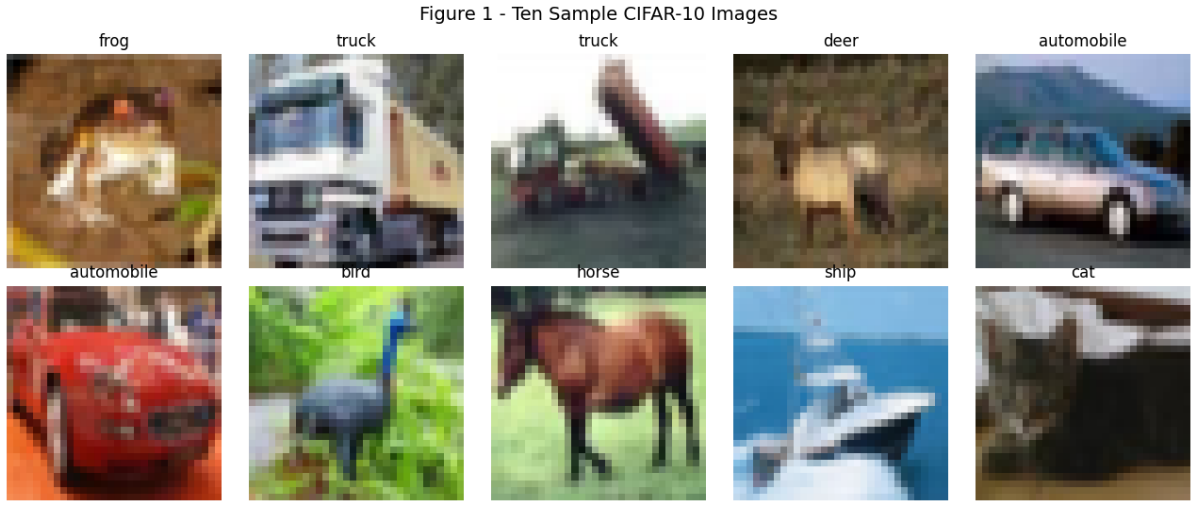


Figure 1: Ten sample CIFAR-10 images with their class labels.

Inference (Figure 1). The ten samples confirm that CIFAR-10 images are very low resolution (32×32) and cover visually diverse natural categories. Objects fill most of the frame but carry very little fine texture, so a network must rely on coarse shape and colour cues. This low resolution is exactly why the images have to be upsampled before being fed to an ImageNet-pretrained backbone.

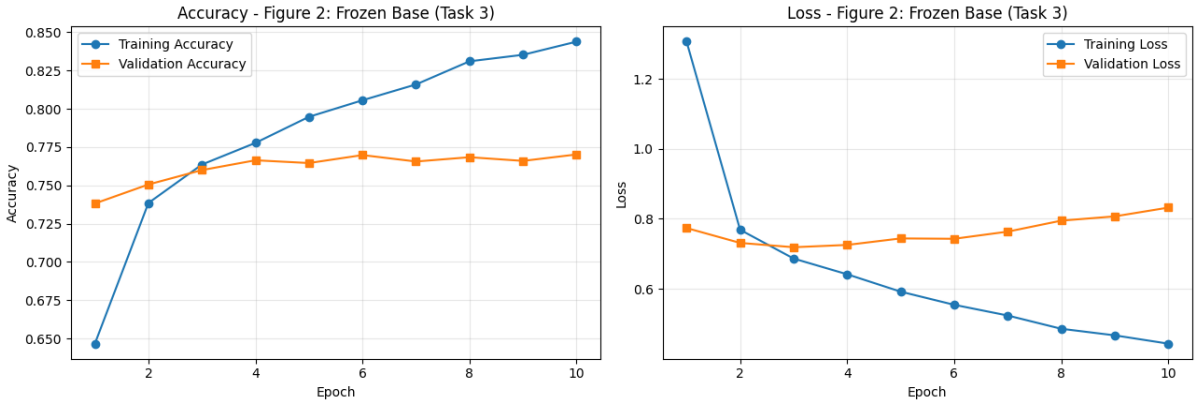


Figure 2: Training and validation accuracy and loss for the frozen-base phase (Task 3, 10 epochs).

Inference (Figure 2). Accuracy rises very steeply in the first two epochs and then flattens, which is the classic transfer-learning signature — the ImageNet features are already informative, so only the small Dense head has to be fitted. Validation accuracy plateaus near 0.77 while training accuracy keeps climbing to 0.84, and the validation loss starts rising after epoch 4: the 14.7 M frozen convolution weights cannot overfit, so the gap that appears comes entirely from the 134 K head parameters.

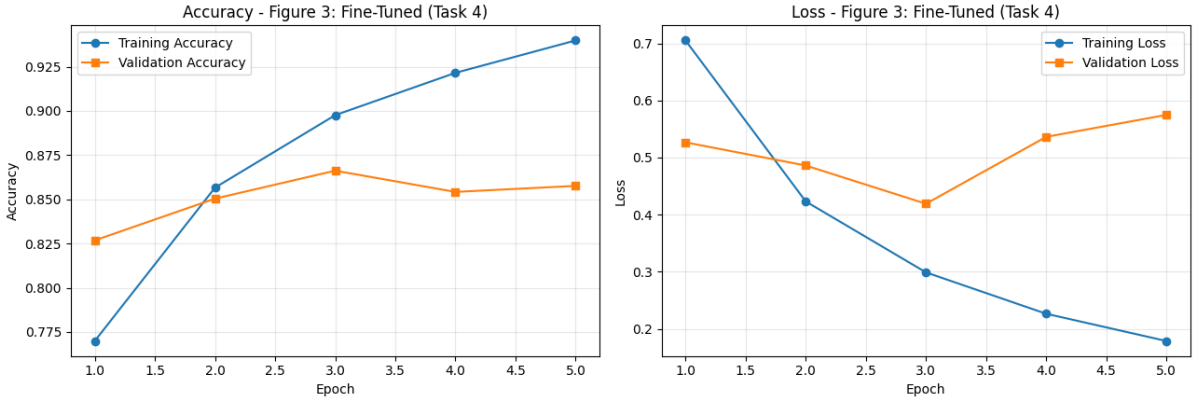


Figure 3: Training and validation accuracy and loss during fine tuning (Task 4, 5 epochs, block5 unfrozen).

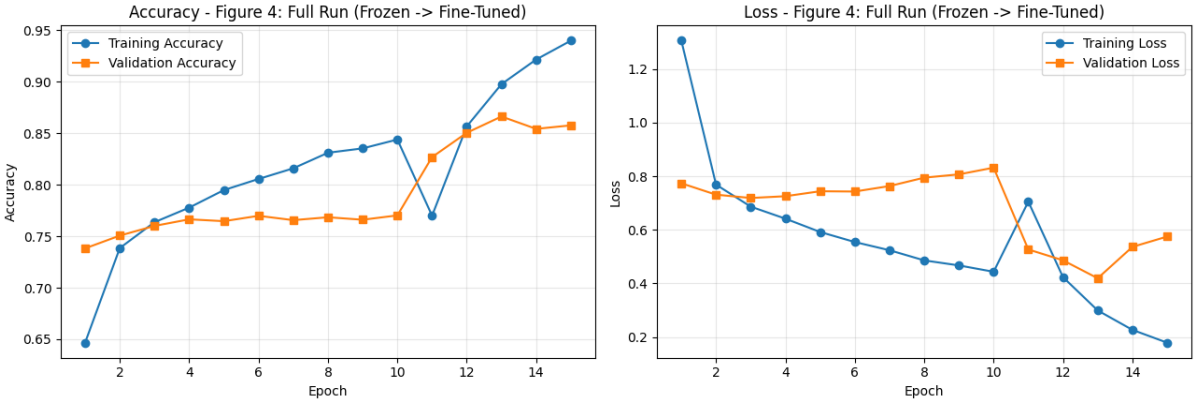


Figure 4: Combined view of the full 15-epoch run: epochs 1–10 frozen base, epochs 11–15 fine tuning.

Inference (Figures 3 and 4). Unfreezing `block5` produces an immediate jump in both training and validation accuracy — validation goes from 0.770 to 0.827 in a single epoch — because the last convolution block can now specialise its high-level ImageNet filters to CIFAR-10's ten classes. The train/validation gap then widens sharply (0.94 train vs 0.86 validation by epoch 5): with 7.2 M trainable weights and only 45,000 small images the model begins to overfit, which is exactly why the learning rate is reduced and only five epochs are used.

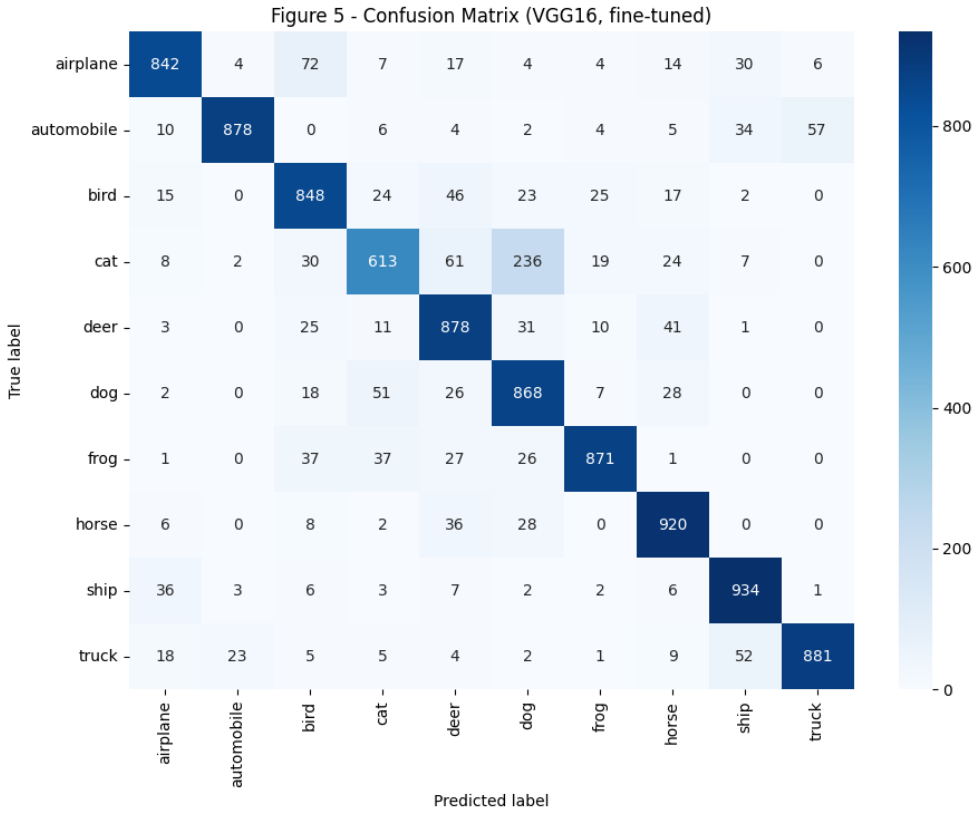


Figure 5: Confusion matrix of the fine-tuned VGG16 model on the 10,000 test images.

Inference (Figure 5). The matrix is strongly diagonal, confirming good overall separation. The residual errors are concentrated in the animal block — **cat vs dog** above all (cat recall is only 0.613, and dog's precision of 0.710 shows it absorbs those errors) and **deer / horse / bird** — because these classes share texture, colour and pose, and at 32×32 the distinguishing details simply are not present. Vehicle classes (automobile, truck, ship, airplane) are recognised most reliably, since their rigid geometric shapes survive the downsampling.

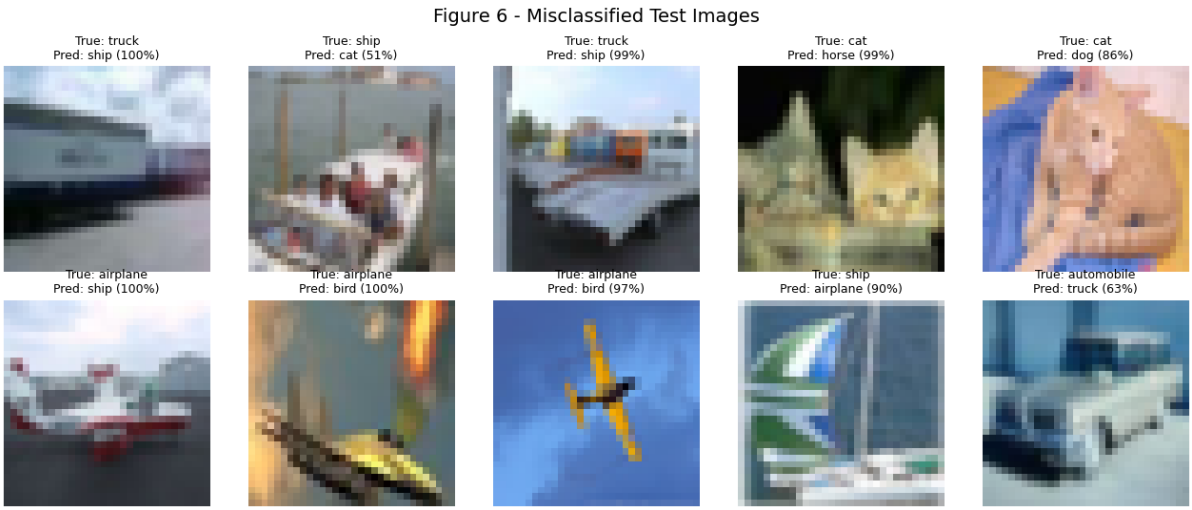


Figure 6: Ten randomly selected misclassified test images with true label, predicted label and softmax confidence.

Inference (Figure 6). Most failures are genuinely hard even for a human at 32×32: animals photographed from unusual angles, heavily blurred objects, or images where the object occupies a small part of a cluttered background. Several errors carry a high softmax confidence, which shows the network is confidently wrong rather than uncertain — a symptom of the low input resolution rather than of insufficient training.

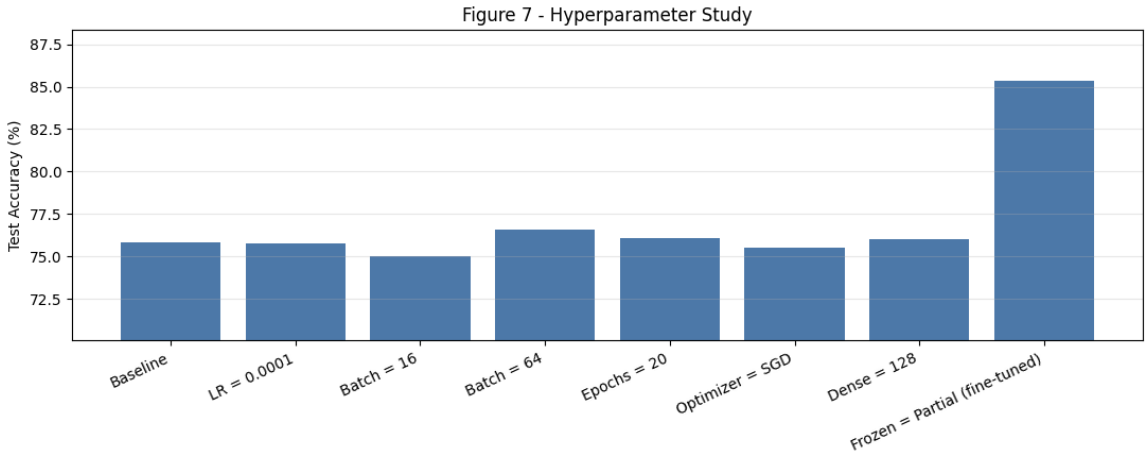


Figure 7: Test accuracy for each hyperparameter variant.

Inference (Figure 7). Dropping the learning rate to 0.0001 slows convergence, so after the same 10 epochs the head is still under-fitted; SGD with momentum is likewise slower than Adam at an identical learning rate. Batch size mainly trades speed against gradient noise — batch 16 takes 89.8 s and actually scores lowest, batch 64 finishes in 25.6 s. Doubling the epochs or halving the Dense units changes test accuracy by less than one point, confirming that the frozen features, not the head capacity, are the bottleneck. **Partial freezing (fine tuning) is by far the most effective change**, worth roughly ten accuracy points against every other variant's fraction of a point. That is the central conclusion of the experiment.

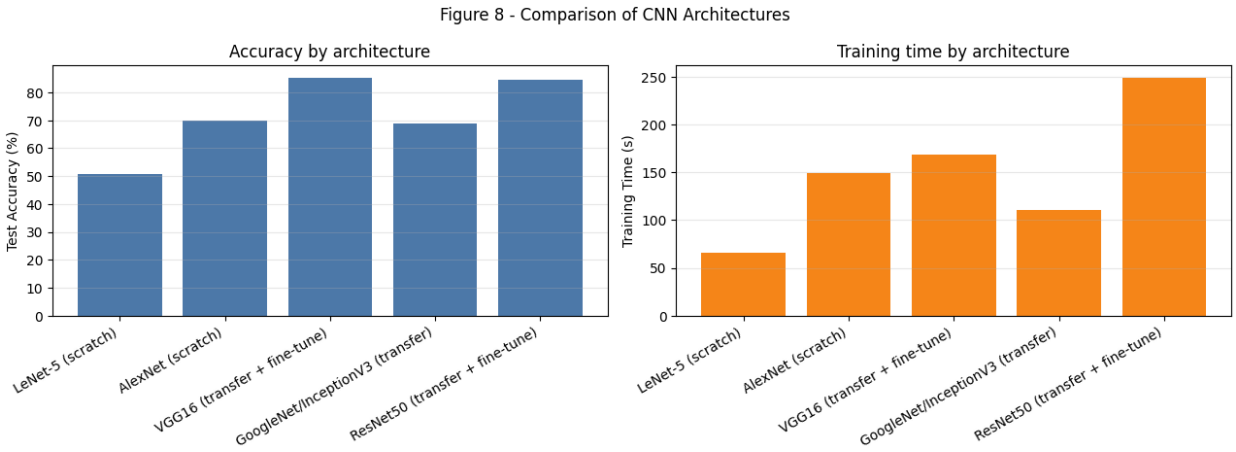


Figure 8: Test accuracy and training time for each architecture.

Inference (Figure 8). Accuracy broadly tracks the historical evolution of CNNs. LeNet-5 (83 K parameters, tanh, average pooling) saturates at 50.7 % because its capacity and receptive field are far too small for natural colour images. AlexNet, with ReLU, dropout and five convolution layers, reaches 69.9 % but still has to learn everything from scratch on only 45,000 images. The two fine-tuned pretrained backbones overtake both by roughly 15 points while training for a comparable number of epochs — this is the value of transfer learning. Fine-tuned models also cost the most time, because gradients must flow through the unfrozen convolution block.

19. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy (final epoch)	0.9398
Testing Accuracy (before fine tuning)	0.7560
Testing Accuracy (after fine tuning)	0.8533
Precision (weighted)	0.8581
Recall (weighted)	0.8533
F1-score (weighted)	0.8527
Training Time — Task 3 (frozen)	79.8 s
Training Time — Task 4 (fine tuning)	88.9 s
Training Time — Total	168.7 s
Total Parameters	14,848,586
Trainable Parameters (fine-tuning phase)	7,213,322

The dataframe screenshot is Figure **??**.

18.2 Comparison of CNN Architectures

Each architecture below was actually trained in this experiment, so the table contains measured numbers. LeNet-5 and AlexNet were trained from scratch on the raw 32×32 images for 10 epochs; VGG16 and ResNet50 used transfer learning followed by fine tuning of the last block; InceptionV3 (GoogleNet) used transfer learning with a frozen base only, at 96×96 input because InceptionV3 requires at least 75×75.

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5 (scratch)	83,126	50.67	66.3
AlexNet (scratch)	8,048,394	69.90	149.7
VGG16 (transfer + fine-tune)	14,848,586	**85.33**	168.7
GoogleNet / InceptionV3 (transfer, frozen)	22,329,898	68.85	110.8
ResNet50 (transfer + fine-tune)	24,114,826	84.58	249.3

Two results deserve comment.

- **InceptionV3 scores lowest of the pretrained backbones** because it was only run with a frozen base — its frozen-base score of 68.85 % is in fact comparable to VGG16's frozen-base score of 75.60 % before fine tuning, and it never received the fine-tuning step that gave the other two their large gains. Inception's aggressive early downsampling also costs more at small input sizes than VGG's uniform stacks.
- **ResNet50 starts stronger but ends slightly lower than VGG16** (frozen base 78.07 % vs 75.60 %, fine tuned 84.58 % vs 85.33 %). Its fine-tuning run overfits harder — training accuracy reaches 0.976 while validation stalls at 0.852 — because unfreezing `conv5` exposes far more parameters, and batch-normalisation statistics adapt poorly to a small 64×64 dataset.

The dataframe screenshot is Figure **??**.

20. Discussion Questions

1. Why is AlexNet considered a breakthrough in deep learning?

AlexNet won ILSVRC-2012 by cutting the top-5 error from about 26 % to about 15 %, a margin no hand-crafted feature pipeline could match. It proved that deep CNNs trained on large data with GPUs are practical: it introduced ReLU activations (removing the saturation of tanh/sigmoid and speeding convergence several-fold), dropout regularisation, overlapping max pooling, data augmentation and multi-GPU training. It effectively started the modern deep-learning era in computer vision.

2. Why does VGG16 use only 3×3 convolution filters?

Stacking three 3×3 convolutions gives the same 7×7 effective receptive field but uses $3 \times (3^2C^2) = 27C^2$ parameters instead of $7^2C^2 = 49C^2$ — fewer weights and less computation. Each extra layer also inserts an additional non-linearity, so the stack is more discriminative than one large filter. Uniform 3×3 blocks additionally make the architecture regular and easy to scale in depth.

3. Explain the advantages of the Inception module.

The Inception module applies 1×1, 3×3 and 5×5 convolutions and max pooling **in parallel** and concatenates the outputs, so the network extracts multi-scale features at every stage instead of committing to a single kernel size. The 1×1 "bottleneck" convolutions placed before the larger kernels reduce the channel depth first, cutting parameters and FLOPs dramatically — GoogleNet needs only about 6.8 M parameters versus VGG16's 138 M while being more accurate. The result is higher computational efficiency and better classification performance.

4. What is the purpose of residual learning?

Very deep plain networks suffer from vanishing gradients and from *degradation* (accuracy gets worse when layers are added). ResNet reformulates each block to learn the residual $F(x) = H(x) - x$, so the output is $F(x) + x$. The identity shortcut gives gradients a direct path back to earlier layers, and a block can trivially learn the identity mapping if the extra depth is not useful. This makes 50-, 101- and 152-layer networks trainable and speeds up convergence.

5. Differentiate LeNet and ResNet.

	LeNet-5 (1998)	ResNet50 (2015)
Depth	7 layers	50 layers
Parameters	≈60 K	≈25.6 M
Activation	tanh / sigmoid	ReLU
Pooling	Average pooling	Max pooling + Global Average Pooling
Normalisation	None	Batch Normalisation
Connections	Strictly sequential	Residual / skip connections
Target data	32 × 32 grayscale digits (MNIST)	224 × 224 colour ImageNet, 1000 classes
Purpose	Proof that CNNs work	Enabling very deep networks

6. What is Transfer Learning?

Transfer learning reuses a model already trained on a large source dataset (e.g. ImageNet) for a new, usually smaller, target task. The pretrained convolution layers already encode generic visual features — edges, textures, object parts — so we remove the original classifier, freeze the convolutional base, attach a new classifier head suited to the new label set, train it, and optionally fine tune the top convolution layers. It gives higher accuracy on small datasets with far less training time and cost.

7. Why is fine tuning required?

Frozen ImageNet features are generic; the deepest layers encode ImageNet-specific concepts that may not match the target domain (here, low-resolution CIFAR-10 objects). Unfreezing the last convolution block lets those high-level filters re-specialise to the new classes, which added **9.73 percentage points** in Task 4 (0.7560 → 0.8533). A reduced learning rate is essential so that the pretrained weights are refined rather than destroyed.

8. Explain the difference between dilated convolution and transpose convolution.

	Dilated (atrous) convolution	Transpose convolution
Effect on size	Keeps the resolution the same	Increases (upsamples) the resolution
Mechanism	Inserts gaps (dilation rate $D > 1$) between kernel elements	Learnable upsampling — inserts zeros / strides, then convolves
Parameters	Same as a normal kernel; receptive field grows for free	Adds learnable weights
Purpose	Enlarge the receptive field / capture context	Reconstruct or generate higher-resolution maps
Typical use	Semantic segmentation, medical and satellite imagery	Autoencoders, GANs, super-resolution, segmentation decoders

9. Why do pretrained models converge faster?

Their weights already sit in a good region of the loss surface: the low- and mid-level filters (edges, corners, textures, object parts) are already correct and need little or no change. Training therefore starts from a much lower loss, only the small head has to be learned from random initialisation, and the gradients are better conditioned. In this experiment the frozen-base head reached 0.7382 validation

accuracy after a *single* epoch, whereas AlexNet trained from scratch needed all ten epochs to reach 0.7128. Random initialisation has to rediscover all of that generic structure, which needs many more epochs and much more data.

10. Compare the computational complexity of LeNet and ResNet.

LeNet-5 has about 60 K parameters (83 K in the CIFAR version used here, because the input has three colour channels) and roughly 0.3–0.5 MFLOPs per 32×32 image — it can be trained on a CPU in minutes. ResNet50 has about 25.6 M parameters ($\approx$425$\times$ more) and about 3.8 GFLOPs per 224×224 image (roughly four orders of magnitude more arithmetic), needing GPUs and hours-to-days of training on ImageNet. The trade-off is capacity: LeNet handles simple grayscale digits, while ResNet models 1000 complex natural-image classes. This is visible in the 18.2 table, where LeNet trains fastest (66.3 s) but scores lowest (50.67 %), and ResNet50 takes the longest (249.3 s) for near-best accuracy.

21. Additional Exercises

#	Exercise	Where it is done
1	Implement transfer learning using VGG16	Sections 15.2–15.5 (main model)
2	Repeat the experiment using ResNet50	Section 18.2; frozen base 78.07 %, fine tuned 84.58 % (+6.51 points)
3	Compare Adam and SGD optimizers	Section 16; Adam 75.80 % vs SGD 75.50 % at the same learning rate
4	Compare training with frozen layers and fine tuning	Section 15.4 and Section 16 (Frozen = All vs Partial): 75.60 % vs 85.33 %
5	Evaluate the model using Precision, Recall and F1-score	Section 15.5 and Section 18.1
6	Compare the performance of LeNet, AlexNet and ResNet	Section 18.2 and Figure 8: 50.67 % / 69.90 % / 84.58 %

22. Conclusion

Transfer learning with an ImageNet-pretrained VGG16 reached 85.33 % test accuracy on CIFAR-10, against 50.67 % for LeNet-5 and 69.90 % for AlexNet trained from scratch for the same ten-epoch budget. Pretrained convolutional features therefore transfer well even to a very different, low-resolution dataset.

Fine tuning the last convolution block at a ten-times lower learning rate added 9.73 percentage points on top of the frozen-base result, and was by a wide margin the most effective change in the whole hyperparameter study — every other variant (learning rate, batch size, epochs, optimizer, Dense units) moved test accuracy by less than 1.6 points. The deepest ImageNet filters clearly do need to be re-specialised for the target domain.

The architecture comparison reproduces the historical trend: deeper networks built from better blocks (3×3 stacks, Inception modules, residual connections) achieve higher accuracy. The main practical caveat observed is that a stronger backbone is not automatically better — ResNet50 overfits faster than VGG16 on 45,000 small images, and InceptionV3 without fine tuning falls behind both.

23. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.

2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: `https://www.tensorflow.org`
8. Keras Documentation: `https://keras.io`